

Aula 09 - Simulação de Sensores Digitais e Analógicos

Descrição: Simulação de leitura de sensores (temperatura, luminosidade, presença) e condicionamento de sinal.

Introdução

A aula de hoje é o ponto de partida para a criação de sistemas IoT, marcando o início da nossa estação de telemetria. O objetivo é compreender como um microcontrolador interage com o mundo real. Para que um sistema tome decisões, ele primeiro precisa "sentir" o ambiente. Nesta aula, exploraremos como capturar estímulos físicos (luz, calor, movimento e distância) e transformá-los em dados computacionais dentro da linguagem C. A lógica da aula seguirá uma progressão clara: entender o conceito físico, compreender o componente no simulador e, em seguida, aplicar o código em blocos isolados antes do desafio final.

Etapa 1: O Universo dos Sensores

Para que um sistema de software interaja com o mundo físico, utilizamos **sensores**, que são transdutores capazes de converter grandezas físicas em sinais elétricos. Na arquitetura do Arduino Uno (ATmega328P), dividimos esses sensores em duas categorias principais de leitura:

1. **Sensores Digitais:** Trabalham com lógica booleana discreta. O sinal elétrico é interpretado apenas como **HIGH** (Nível Alto, 5V, ou 1) e **LOW** (Nível Baixo, 0V, ou 0).
 - **Exemplo:** Um sensor de presença PIR. Ele responde de forma binária: há movimento (**HIGH**) ou não há movimento (**LOW**).
2. **Sensores Analógicos:** O mundo real não é feito apenas de 0 e 1; ele é contínuo. Sensores analógicos fornecem uma variação contínua de tensão entre 0V e 5V. Para que o C entenda isso, o Arduino possui um Conversor Analógico-Digital (ADC) de 10 bits, que "fatia" essa tensão em 1024 degraus. Portanto, uma leitura analógica no pino **A0** entregará um número inteiro variando de 0 a 1023.
 - **Exemplo:** O sensor de temperatura LM35 e o resistor dependente de luz (LDR).
3. **Sensores Baseados em Tempo de Pulso:** Operam com sinais digitais, mas a informação da grandeza física não está no nível lógico imediato (0 ou 1), e sim no tempo de duração em que um pulso permanece em nível lógico alto.
 - **Exemplo:** O sensor ultrassônico HC-SR04. Ele emite um pulso de som e aciona um pino digital durante o tempo exato em que a onda sonora viaja até um obstáculo e retorna (eco), permitindo calcular a distância.

Prática 1: Leitura Bruta dos Sensores

Nesta fase, vocês devem acessar o Tinkercad e inserir:

- um Arduino Uno,
- um LDR,
- um sensor de temperatura LM35 e
- um sensor PIR.

Montagem base: Alimentar os sensores com 5V e **GND**. Ligar o sinal do PIR no pino digital **2**, o sinal do LM35 no **A1** e o sinal do LDR no **A0**.

Bloco de Código 1 - Leitura básica de sensores:

```
C
// Definicao dos pinos
const int PINO_LDR = A0; // Entrada analogica
const int PINO_TEMP = A1; // Entrada analogica
const int PINO_PIR = 2; // Entrada digital

void setup() {
    Serial.begin(9600); // Inicia a comunicacao serial para inspecao
    pinMode(PINO_PIR, INPUT); // Configura pino digital como entrada
}

// Você deve rodar este laço e observar os dados crus no Monitor Serial
void loop() {
    int leituraLdrBruta = analogRead(PINO_LDR); // Retorna 0 a 1023
    int leituraTempBruta = analogRead(PINO_TEMP); // Retorna 0 a 1023
    int presenca = digitalRead(PINO_PIR); // Retorna 0 ou 1

    Serial.print("LDR Bruto: "); Serial.println(leituraLdrBruta);
    Serial.print("TEMP Bruto: "); Serial.println(leituraTempBruta);
    Serial.print("PIR: "); Serial.println(presenca);
    delay(1000);
}
```

Explicação do código:

O Bloco de Código 1 tem a função de ler as informações "cruas" dos sensores e exibi-las na tela do computador, dividindo-se em três partes principais:

1. Definição dos Pinos (Nomeando as conexões):

As primeiras linhas apenas dão nomes aos pinos do Arduino para facilitar o uso no código. Em vez de memorizar que o sensor de luz (LDR) está conectado no pino **A0**, criamos a constante **PINO_LDR** para representá-lo.

2. Configuração Inicial (**setup**):

Este bloco roda apenas uma vez quando o Arduino é ligado.

- O comando **Serial.begin(9600)** abre o canal de comunicação entre o Arduino e o computador para que possamos visualizar os dados no Monitor Serial.
- O comando **pinMode(PINO_PIR, INPUT)** avisa o microcontrolador que o pino digital 2 servirá como uma porta de entrada (**INPUT**) para receber a informação do sensor de presença.

3. Leitura e Exibição Contínua (**loop**):

Este bloco trabalha em um ciclo infinito, executando as instruções repetidamente.

- **Lendo o mundo analógico:** O comando `analogRead` "fatia" a variação contínua de tensão do sensor de luz e de temperatura, entregando números inteiros brutos que vão de 0 a 1023.
- **Lendo o mundo digital:** O comando `digitalRead` verifica o sensor de presença, que trabalha de forma binária, entregando apenas os estados 0 (sem movimento) ou 1 (com movimento).
- **Mostrando os dados:** Os comandos `Serial.print` e `Serial.println` imprimem esses números recém-capturados na tela.
- **Pausa:** O `delay(1000)` manda o sistema congelar por 1 segundo (1000 milissegundos) antes de recomençar a leitura.

Para visualizar os resultados:

- Clique no botão "**Código**" no canto superior direito e, em seguida, clique no botão "**Monitor Serial**" (ou *Serial Monitor*) que aparece na parte inferior da tela de código.
- Inicie a simulação (*Iniciar Simulação*) para ver os números subindo em tempo real.

Atenção: Você perceberá que o valor do LM35 impresso (ex: 153) não representa a temperatura em graus Celsius. É aqui que entra o próximo conceito.

Etapa 2: Condicionadores de Sinal

O dado "bruto" (0 a 1023) gerado pelo ADC não tem utilidade direta para o usuário ou para lógicas avançadas de software. Precisamos de **condicionamento de sinal**, que pode ser tanto uma adequação no hardware (elétrica) quanto no software (matemática).

1. Condicionamento Elétrico (Hardware):

- Um LDR altera sua resistência com a luz, mas o Arduino lê *tensão* (Volts). Para converter a variação de resistência em variação de tensão, utilizamos um circuito chamado **Divisor de Tensão**, associando o LDR a um resistor (ex: 10 k Ω) conectado ao `GND` (resistor *pull-down*).

2. Condicionamento Matemático (Software):

- **Temperatura:** O sensor LM35 gera 10mV para cada grau Celsius. Como o ADC divide 5V em 1024 partes, nossa constante de conversão de unidade ADC para Celsius é calculada pela base $(5.0 / 1023.0) * 100.0$, o que nos dá o fator `0.488758`.
- **Distância Ultrassônica:** O sensor HC-SR04 emite um pulso sonoro e mede o tempo de retorno em microssegundos com a instrução `pulseIn()`. Para converter esse tempo em centímetros, dividimos a duração por 58.

Prática 2: Implementando Funções de Conversão

Vocês vão apagar o bloco de código anterior e focar apenas na construção lógica das funções em C que transformam dados elétricos em grandezas físicas padronizadas.

Montagem base: Adicionar o sensor Ultrassônico HC-SR04 ao circuito (Trigger no pino `8`, Echo no pino `7`).

Bloco de Código 2 - Condicionamento Lógico de sinal:

```
C
// 1. Constantes do Bloco 2
const int PINO_TRIG = 8;
const int PINO_ECHO = 7;
const float BASE_CELSIUS = 0.488758;

// 2. Funcoes do Bloco 2
float condicionarTemperatura(int leituraBruta) {
    return (float)leituraBruta * BASE_CELSIUS;
}

int condicionarLuminosidade(int leituraBruta) {
    return (int)((leituraBruta / 1023.0) * 100.0);
}

long lerDistanciaUltrassonica() {
    digitalWrite(PINO_TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(PINO_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(PINO_TRIG, LOW);

    long duracaoDoPulso = pulseIn(PINO_ECHO, HIGH); // Captura a largura do pulso[cite: 1]
    return duracaoDoPulso / 58;
}

// 3. O Motor de Execucao (O que faltava para rodar)
void setup() {
    Serial.begin(9600); // Habilita o Monitor Serial
    pinMode(PINO_TRIG, OUTPUT); // Configura o pino de disparo
    pinMode(PINO_ECHO, INPUT); // Configura o pino de escuta
}

void loop() {
    // Simulando leituras brutas (apenas para validar a conversao matematica)
    int simulacaoLuzBruta = 512; // Valor ficticio (metade de 1023)
    int simulacaoTempBruta = 50; // Valor ficticio

    // Chamando as funcoes e armazenando os retornos
    int luzReal = condicionarLuminosidade(simulacaoLuzBruta);
    float tempReal = condicionarTemperatura(simulacaoTempBruta);
    long distanciaReal = lerDistanciaUltrassonica();

    // Imprimindo os resultados no console
```

```
Serial.print("Luz Simulada: "); Serial.print(luzReal); Serial.println("%");
Serial.print("Temp Simulada: "); Serial.print(tempReal); Serial.println(" C");
Serial.print("Distancia Real (Sensor: "); Serial.print(distanciaReal); Serial.println(" cm");
Serial.println("-----");

delay(1500); // Pausa para facilitar a visualizacao
}
```

Explicação do Código: Este bloco de código atua como um "motor de testes" isolado e um "tradutor" de sinais. Ele prova que a matemática das suas funções está correta antes de você conectar todos os componentes físicos. A lógica se divide nas seguintes etapas:

- 1. Constantes e O Tradutor (A Lógica):** A variável `BASE_CELSIUS` armazena o fator exato (0.488758) para o LM35. As funções `condicionarTemperatura` e `condicionarLuminosidade` usam a matemática (multiplicação e regra de três) para transformar números elétricos em graus e porcentagem. A função `lerDistanciaUltrassonica` controla os pulsos do pino TRIG e usa a velocidade do som (dividindo o tempo por 58) para calcular centímetros exatos no pino ECHO.
- 2. Preparação do Sistema (setup):** O comando `Serial.begin(9600)` liga o Monitor Serial. Os comandos `pinMode` preparam exclusivamente os pinos do sensor ultrassônico (`OUTPUT` para disparar o som e `INPUT` para escutar).
- 3. Dados Fictícios de Teste (loop):** Como não montamos LDR e LM35 neste teste, o código injeta valores elétricos "inventados" (512 para luz e 50 para temperatura) diretamente nas funções matemáticas, validando o condicionamento independentemente do hardware.
- 4. Leitura Ultrassônica Real:** Diferente da luz e temperatura, a distância é testada na prática. A função interage com o sensor HC-SR04 real na tela do simulador.
- 5. Exibição e Ritmo:** Os comandos `Serial.print` organizam e escrevem no console os resultados finais (Luz/Temperatura simuladas e Distância real). O `delay(1500)` pausa o sistema para facilitar a leitura.

Etapas 3: O Desafio do Protagonismo

Nesta etapa prática você assume o papel de desenvolvedor de sistemas, consolidando os blocos lógicos isolados das etapas anteriores em um único firmware funcional. A transição ocorre da aprendizagem guiada para a resolução autônoma de problemas.

1. Dinâmica da Atividade

- Unificação de Código:** Você inicia um *sketch* em branco no ambiente do Tinkercad. Ele deve declarar globalmente os pinos (`PINO_LDR`, `PINO_TEMP`, `PINO_PIR`, `PINO_TRIG`, `PINO_ECHO`) e transcrever as funções de conversão matemática (`condicionarTemperatura`, `condicionarLuminosidade`, `lerDistanciaUltrassonica`) trabalhadas na Etapa 2.
- Construção do Fluxo Principal:** Utilizando a estrutura básica da linguagem C em microcontroladores,

você codifica a função `setup()` para inicializar a comunicação serial e definir os modos dos pinos (`INPUT` ou `OUTPUT`). Em seguida, desenvolve a rotina contínua dentro da função `loop()`.

- **Aplicação de Lógica Condicional:** Utilize as estruturas de controle de decisão (`if/else`) para analisar os dados condicionados e determinar o estado de segurança do ambiente com base no sensor PIR.

2. Especificações Técnicas Requeridas

- Os valores brutos obtidos por `analogRead()` e `digitalRead()` não podem ser impressos diretamente na tela. Eles devem ser armazenados em variáveis locais e, obrigatoriamente, passados como parâmetros para as funções de condicionamento.
- A avaliação do sensor PIR deve verificar o estado lógico. Se o pino estiver recebendo 5V, a leitura será interpretada como `HIGH` (movimento), caso contrário, será `LOW` (repouso).
- O sistema deve conter um intervalo de tempo (`delay()`) no final do ciclo principal, estabilizando a taxa de atualização dos dados no Monitor Serial.

3. O Entregável

Você deve submeter o link de compartilhamento do projeto funcional no simulador Tinkercad. A entrega é composta por três frentes de avaliação:

- **Circuito Físico Virtual:** Montagem dos quatro sensores na *protoboard*, garantindo o barramento de alimentação (5V e `GND`) correto para o HC-SR04, PIR e LM35, além do divisor de tensão resistivo devidamente conectado ao LDR.
- **Código-Fonte em C Polido:** O código deve apresentar indentação correta, uso de funções modulares para os cálculos e comentários pontuais (utilizando `//`) para explicar as lógicas de conversão e estruturação de dados.
- **Console de Saída Formatado (Monitor Serial):** O sistema deve processar as grandezas e imprimir uma linha limpa de *telemetria* a cada ciclo, formatando os dados condicionais exatamente como no exemplo abaixo:

Temp: 25.40 C | Luz: 85% | Dist: 120 cm | Status: Área Segura

(Saída gerada quando o valor do PIR for LOW)

Temp: 25.40 C | Luz: 85% | Dist: 120 cm | Status: ALERTA: Presença Detectada

(Saída gerada quando o valor do PIR for HIGH)

Encerramento e Gancho para a próxima aula

Parabéns pela conclusão do núcleo de aquisição de dados! Hoje o nosso sistema ganhou olhos, pele e ouvidos, convertendo grandezas físicas complexas em variáveis numéricas que nosso software em C domina perfeitamente. Vocês perceberam como condicionar os sinais é essencial para obtermos dados confiáveis.

Porém, um sistema que apenas "sente" o ambiente é passivo. Na **Aula 10 - Simulação: Atuadores e Modulação PWM**, daremos vida ao projeto. O código unificado que vocês construíram agora será a base de decisão. Utilizaremos o percentual de luz que acabamos de calcular para movimentar as pás de um servomotor de forma contínua, modulando sua posição. A detecção de presença acionará a comutação de relés de potência para ligar lâmpadas reais, e o limiar de temperatura fará o chaveamento de transistores para controle de velocidade de motores CC via PWM. O código de leitura da Aula 09 vai disparar a ação mecânica da Aula 10! Guardem bem este código, ele é o cérebro da nossa próxima montagem.